

Conformance Checking for Process Improvement

JM0211 Process Mining

Group 8: Rick de Rijk, Thom Verzantvoort, Stefan Vonk, Tycho van Rooij

Table of contents

1 Motivation and Scientific Background	2
2 Conceptual Approach	2
2.1 Overview	2
2.2 Step 1 KPI definition and enrichment	4
2.3 Step 2 Alignment and encoding	4
2.4 Step 3 Classification	4
2.5 Step 4 Rules and log split	4
3 Implementation	4
4 Evaluation Design and Results	5
5 Limitations and Future Work	5
References	5

List of Figures

i Working draft

Page limit on submission: **5 pages excluding references**. Remove this callout before the final PDF. Section ownership while drafting: Rick = KPI / data step; Stefan = alignment and encoding; classification and rules/evaluation still open between Tycho and Thom.

1 Motivation and Scientific Background

Conformance checking compares behaviour recorded in an event log with behaviour permitted by a process model in order to identify commonalities and deviations ([Aalst 2016](#)). Alignment-based conformance checking can localise deviations as *log moves*, representing observed behaviour that cannot be reproduced by the model, and *model moves*, representing modelled behaviour that was not observed.

Deviations from a normative process model are not necessarily associated with poor process performance. Dees et al. ([Dees, Leoni, and Mannhardt 2017](#)) propose a process improvement methodology in which deviations identified through conformance checking are related to a case-level KPI using classification or regression trees. Deviation patterns associated with satisfactory KPI values can subsequently inform process model enhancement, subject to business constraints.

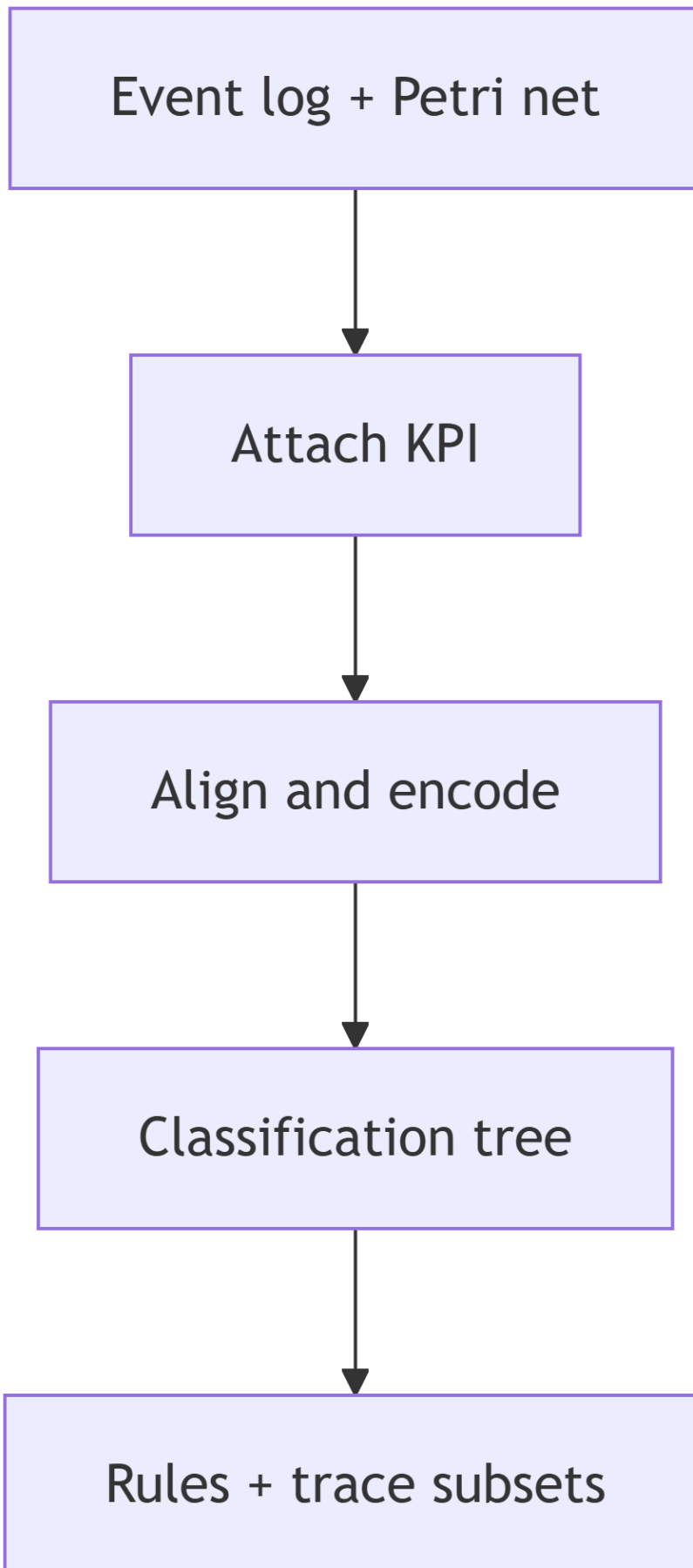
This assignment implements the deviation analysis component of this methodology rather than the complete model repair procedure. For each case, optimal alignments against a Petri net are used to derive deviation features, which are related to case throughput time. Throughput time is defined as the elapsed time between the first and last event of a trace, following the KPI definition used in the UWV case study of Dees et al. ([Dees, Leoni, and Mannhardt 2017](#)).

Within JM0211, neighbouring topics also treat conformance (for example resource or temporal diagnostics) or process performance via predictive and prescriptive monitoring; our topic differs by linking alignment-based deviation features to a case-level KPI specifically for process improvement.

2 Conceptual Approach

2.1 Overview

The artefact takes as input (i) an event log in XES format and (ii) a Petri net in PNML format. It returns (i) a set of classification rules that relate deviation encodings to the KPI, and (ii) for each rule, the subset of traces that satisfy it. The pipeline has four steps.



Overview of the deviation-analysis pipeline.

2.2 Step 1 KPI definition and enrichment

Owner: Rick.

KPI. We use *throughput time* of each trace, following (Dees, Leoni, and Mannhardt 2017): for a trace σ ,

$$\kappa(\sigma) = t(\text{last event of } \sigma) - t(\text{first event of } \sigma).$$

We store $\kappa(\sigma)$ in **days** (elapsed seconds divided by 86400) as a case-level attribute named **throughput_time**. Lower values are better. Empty traces are rejected; a missing **time:timestamp** on an event is treated as an error. A single-event case receives throughput 0.

Required log characteristics. The log must provide a clear case notion and an ordered sequence of events with reliable timestamps. Control-flow activity labels are required for later alignment; resources are not required for this KPI step. The Petri net must be the normative model against which alignments will be computed in the next step.

Output of this step. The same event log, enriched with **throughput_time** on every case (optionally exported as a new XES file). No parameters beyond optional subsampling for development speed.

2.3 Step 2 Alignment and encoding

Owner: Stefan.

For each trace, compute an optimal alignment with the Petri net. Encode each trace by counting, per activity a_i , the number of model moves and the number of log moves. This yields two features per activity (Dees, Leoni, and Mannhardt 2017).

2.4 Step 3 Classification

Owner: Tycho or Thom (not yet assigned).

Train a classification tree on the encoded traces, using **throughput_time** as the dependent variable. Use standard hyperparameter search to select a well-performing tree. *Note:* if the KPI is kept continuous, a regression tree may be the direct analogue of (Dees, Leoni, and Mannhardt 2017); if the course brief is read strictly as a *classification* tree, the KPI must be discretised (for example by a threshold). The group should lock that choice before modelling.

2.5 Step 4 Rules and log split

Owner: Tycho or Thom (not yet assigned).

Derive classification rules from the tree and split the original log so that each rule maps to a compliant subset of traces. These subsets are the second required artefact output.

3 Implementation

Rick (done in code). Python 3 with **pm4py** loads the XES log and PNML net. A small module computes and attaches **throughput_time**, with a CLI entry point (**main.py**) supporting **--sample** and optional **--export** of an enriched XES. Paths are relative to the project root.

Stefan, Tycho, Thom. *To be filled:* alignment API, encoding schema, scikit-learn tree setup, rule extraction, and how to reproduce evaluation scores from the repository README.

4 Evaluation Design and Results

Owner: Tycho or Thom (lead once assigned), with group input.

The brief requires a manually built gold standard and at least precision and recall for (1) optimal alignments per trace and (2) the recovered classification rules. Dataset: BPI Challenge 2017 (Denied) as provided with the assignment; a sample may be used during development, with final numbers on the full log.

Results and figures: pending implementation of steps 2 to 4.

5 Limitations and Future Work

KPI step. Throughput time uses only the first and last timestamps. It ignores waiting-time structure inside the case, calendar effects, and unfinished cases that might appear in a broader log. Discretising the KPI for a classification tree (if required) throws away magnitude information that a regression tree would keep (Dees, Leoni, and Mannhardt 2017).

Broader approach. Further limitations of alignments, tree stability, and gold-standard construction will be added once those stages exist. Future work suggested by (Dees, Leoni, and Mannhardt 2017) includes multi-KPI improvement and incorporating only legally admissible repairs; those sit outside our implemented deviation-analysis scope but remain relevant for process improvement.

References

- Aalst, Wil M. P. van der. 2016. *Process Mining: Data Science in Action*. 2nd ed. Springer. <https://doi.org/10.1007/978-3-662-49851-4>.
- Dees, Marcus, Massimiliano de Leoni, and Felix Mannhardt. 2017. “Enhancing Process Models to Improve Business Performance: A Methodology and Case Studies.” In *On the Move to Meaningful Internet Systems. OTM 2017 Conferences*, 3–22. Springer.